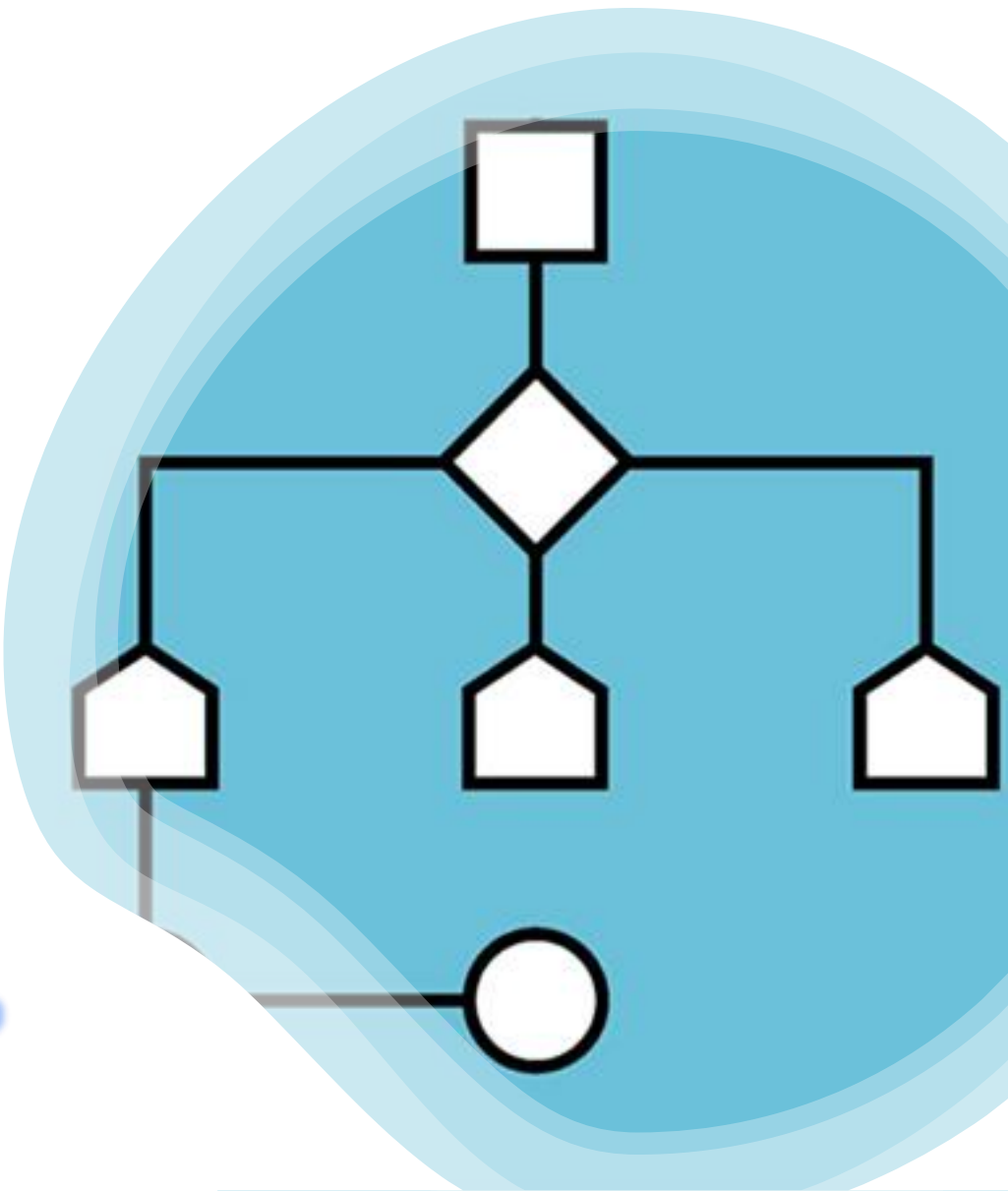


Déroulement du TP

- **Introduction :**
 - Pourquoi trier ?
 - Pas à pas : tris lents et rapides
 - Tris en place et hors place
- **Mise en jambe : Le tri par "sélection" hors place**
- **Les tri par "sélection" en place**
- **Le tri par "insertion"**
- **Bonus du/des cadors : le tri à bulles**

Introduction



Pourquoi trier ?

POURQUOI TRIER ?



C'est une excellente activité de formation et de perfectionnement aux algorithmes.

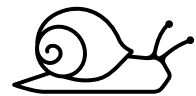


Certains algorithmes comme l'algorithme de recherche dichotomique (ou tri par dichotomie ou *binary search* en anglais) ont besoin de travailler avec des données déjà triées.



C'est un traitement de base lorsque l'on désire analyser, synthétiser des données.

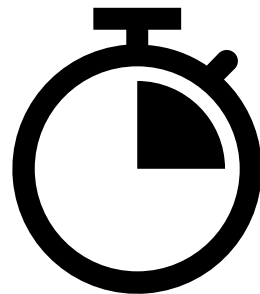
Introduction



Pas à pas : tris lents et rapides

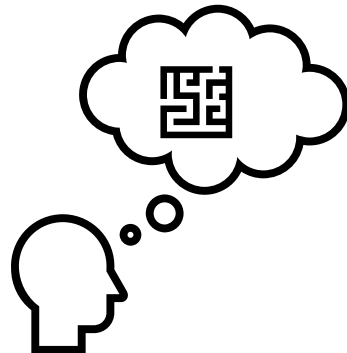
PAS À PAS : ALGORITHMES LENTS & RAPIDES

- Dans ce TP nous aborderons que des algorithmes de tri lent.
- En effet il existe de nombreux algorithmes de tri dont de très performants :
 - Tri rapide (quick sort)
 - Tri fusion
 - Tri par arbre binaire de recherche



PAS À PAS : ALGORITHMES LENTS & RAPIDES

- Cependant ces algorithmes sont plus compliqués à implémenter car ils nécessitent notamment :
 - Des stratégies algorithmiques que vous n'avez pas encore vu : Par exemple et généralement les algorithmes de tri performants scindent les données à trier en plusieurs blocs pour gagner en efficacité.
 - Ces algorithmes de tri peuvent nécessiter des structures de données que vous ne connaissez pas encore : arbre, file, pile, etc.



PAS À PAS : ALGORITHMES LENTS & RAPIDES

- L'objectif de ce TP n'est pas donc la recherche de la performance, mais l'apprentissage.
- Les langages comme Python intègrent nativement des fonctions/méthodes efficaces pour trier leurs listes/tableaux.

Introduction

Tris en place et hors place

TRIS EN PLACE ET HORS PLACE

- Le 1^{er} algorithme que vous implémenterez sera un tri hors place.
- Cela veut dire que vous allez utiliser une 2^e liste initialement vide pour accueillir les valeurs dans l'ordre de tri (tri croissant dans notre cas).
- Cette méthode est loin d'être idéale, car elle consomme de l'espace mémoire !
- Donc, dans les algorithmes qui suivront dans la suite du TP, le tri se fera implicitement sans utiliser une liste vide.
- On parle alors de tri en place, car on travaille directement sur la liste à trier pour minimiser l'espace mémoire consommée.

Mise en jambe : Tri par sélection hors place



MISE EN JAMBE : TRI PAR SÉLECTION EN PLACE

- Pour vous mettre en forme, vous allez procéder avec une version simplifiée du tri dit par "sélection" l'implémentant avec une version en place (utilisation d'une liste pour accueillir les valeurs triées).
- Pour réaliser et valider votre travail on vous fournit un fichier Python initial :
- Chaque algorithme de tri sera implémenté dans une fonction dédiée.
- Le fichier comporte déjà le "squelette" de chacune des fonctions :



```
72 # Fonction de tri sélection hors place :
73 def triSelectionHorsPlace(liste):
74     """Fonction appliquant le tri par sélection hors place à la
75     liste donnée en argument et qui retourne une copie de la liste
76     originale triée."""
77
78     listeTriee = []
79     copyListe = liste[:]
80
81     while(len(copyListe) > 0):
82         # Recherche de l'indice contenant la valeur minimale de liste :
83         iValMin = 0
84         # A COMPLETER
85
86         # Transfert de la valeur minimale dans la liste triée :
87         listeTriee.append(copyListe.pop(iValMin))
88
89     return listeTriee
```

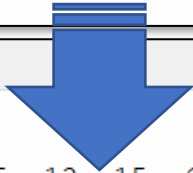
```
92 # Fonction de tri sélection en place :
93 def triSelectionEnPlace(liste):
94     """Fonction appliquant le tri par sélection en place à la
95     liste donnée en argument et qui retourne une copie de la liste
96     originale triée."""
97
98     copyListe = liste[:]
99     N = len(copyListe)
100
101     # Boucle générale (de ... à ...) :
102
103     # Boucle de recherche de la valeur mini (de ... à ...) :
104
105     # Permutation des valeurs :
106
107
108     return copyListe
```

```
111 # Fonction de tri par insertion :
112 def triInsertion(liste):
113     """Fonction appliquant le tri par insertion à la
114     liste donnée en argument et qui retourne une copie de la liste originale
115     triée."""
116
117     copyListe = liste[:]
118     N = len(copyListe)
119
120     # Boucle principale (de ... à ...) :
121
122     # Boucle des permutations (de ... à ... ET
123
124     return copyListe
```

MISE EN JAMBE : TRI PAR SÉLECTION EN PLACE

- Le fichier comporte également 3 fonctions et les instructions nécessaires dans le programme principal pour réaliser des tests sur l'implémentation de vos fonctions :

```
139 # -----
140 # PROGRAMME PRINCIPAL :
141 # -----
142
143 # TEST UNIQUE :
144 # -----
145 testList = createRandList(20)
146 print("La liste test est :\t\t" + str(testList))
147 listeTrie = triSelectionHorsPlace(testList)
148 #listeTrie = triSelectionEnPlace(testList)
149 #listeTrie = triInsertion(testList)
150 print("La liste test après le tri :\t" + str(listeTrie))
151
152
153 # TEST MULTIPLE :
154 # -----
155 print("Multitests : ", multiTest(triSelectionHorsPlace, 50, 1000))
156
```



```
Console x
>>> %Run 'TP tri de listes.py'
La liste test est :          [6, 1, 7, 8, 15, 10, 7, 17, 2, 5, 13, 15, 6, 19, 12, 5, 7, 8, 6, 12]
La liste test après le tri : [6, 1, 7, 8, 15, 10, 7, 17, 2, 5, 13, 15, 6, 19, 12, 5, 7, 8, 6, 12]
Multitests : False
>>>
```

MISE EN JAMBE : TRI PAR SÉLECTION EN PLACE

- L'algorithme du tri par sélection HORS PLACE est simple et peut être décrit par le texte suivante :

Créer une liste vide

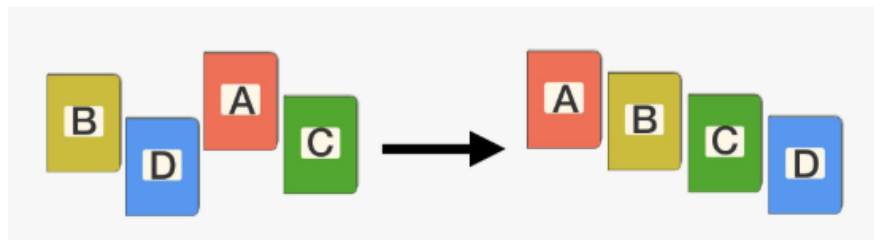
REPETER TANT QUE LA LISTE INITIALE N'EST PAS VIDE :

Rechercher l'indice de la valeur minimale dans la liste initiale

Extraire la valeur minimale de la liste initiale et l'insérer dans le plus petit indice disponible

- Implémenter cet algorithme dans la fonction `triSelectionHorsPlace` et tester-le :

```
72 # Fonction de tri sélection hors place :
73 def triSelectionHorsPlace(liste):
74     """Fonction appliquant le tri par sélection hors place à la
75     liste donnée en argument et qui retourne une copie de la liste
76     originale triée."""
77
78     listeTrie = []
79     copyListe = liste[:]
80
81     while(len(copyListe) > 0):
82         # Recherche de l'indice contenant la valeur minimale de liste :
83         iValMin = 0
84         # A COMPLETER
85
86         # Transfert de la valeur minimale dans la liste triée :
87         listeTrie.append(copyListe.pop(iValMin))
88
89     return listeTrie
```



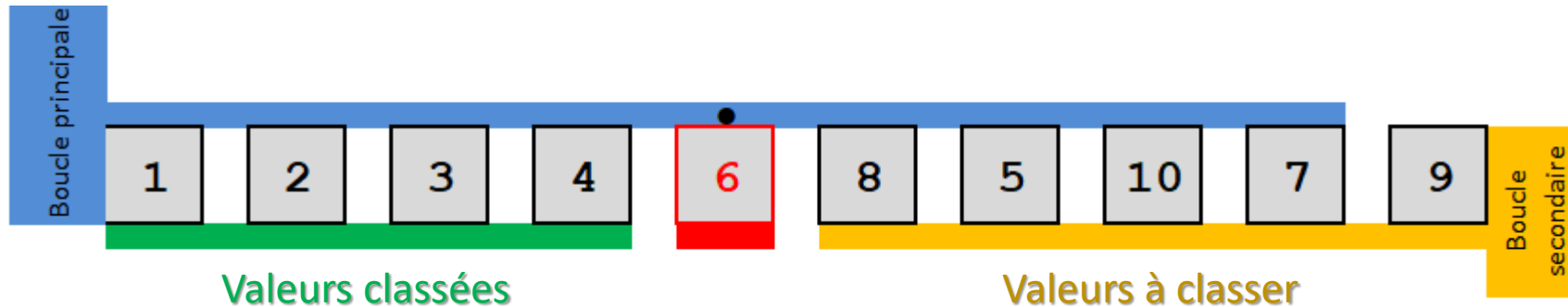
Tri par **selection** en
place

TRI PAR SÉLECTION

- Dans cette partie vous allez implémenter la version EN PLACE du tri par sélection.
- Le principe est identique, c'est-à-dire que vous allez toujours rechercher la plus petite valeur de la liste dans un 1^{er} temps.
- Par contre dans le 2^e temps, vous n'allez pas extraire cette valeur dans une 2^e liste, mais PERMUTER cette valeur avec la valeur située à l'indice 0 au 1^{er} tour.
- Puis on recommencera la recherche de la valeur minimale suivante pour procéder à la permutation avec l'indice 1, puis on recommence avec l'indice 2, etc.

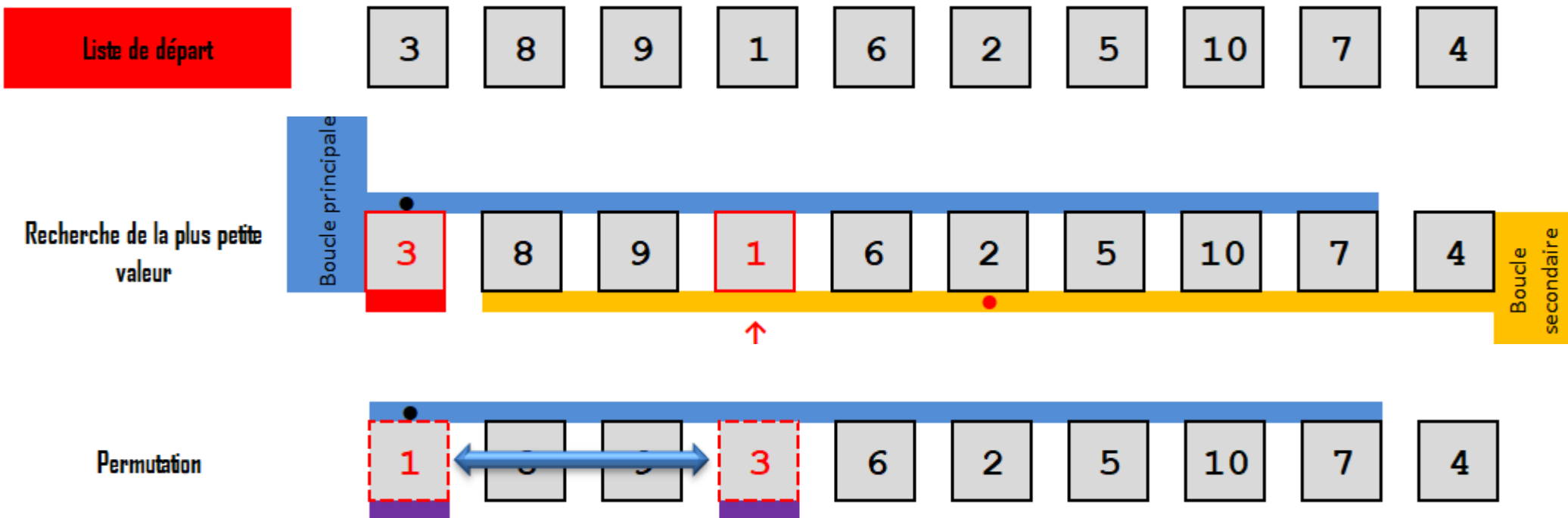
TRI PAR SÉLECTION

- La difficulté est ici de considérer la liste en 2 segments :
 - Le segment de gauche comporte les valeurs déjà classées
 - Le segment de droite comporte les valeurs qui restent à classer



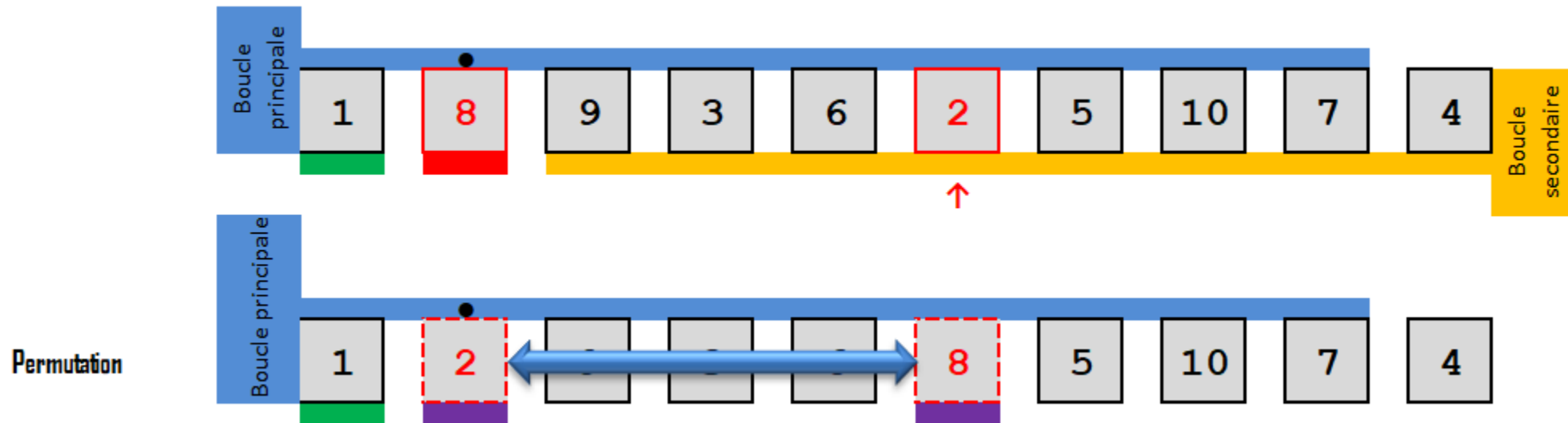
TRI PAR SÉLECTION

- Exemple :



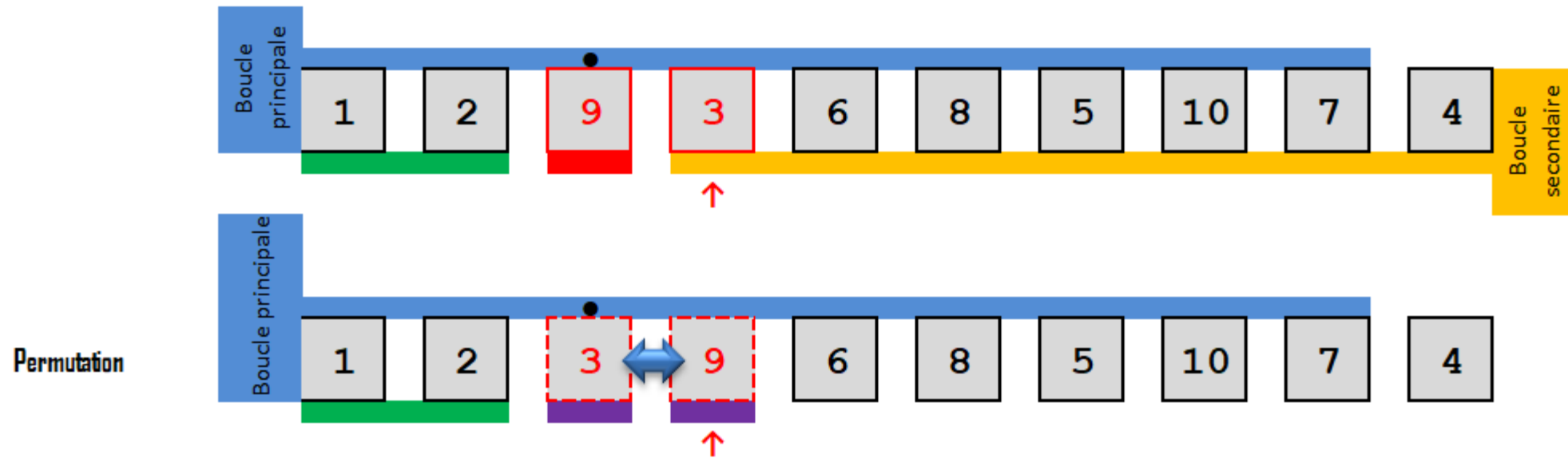
TRI PAR SÉLECTION

- Exemple :



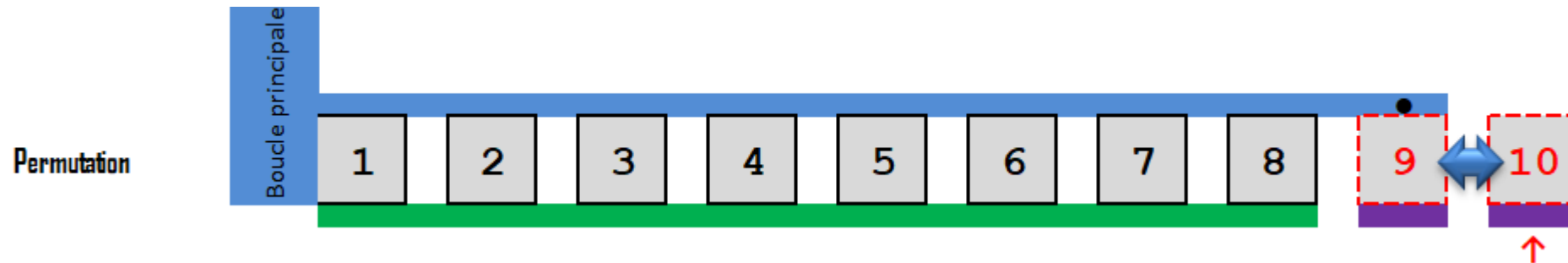
TRI PAR SÉLECTION

- Exemple :

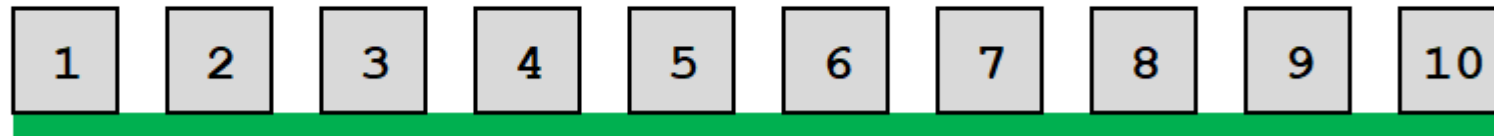


TRI PAR SÉLECTION

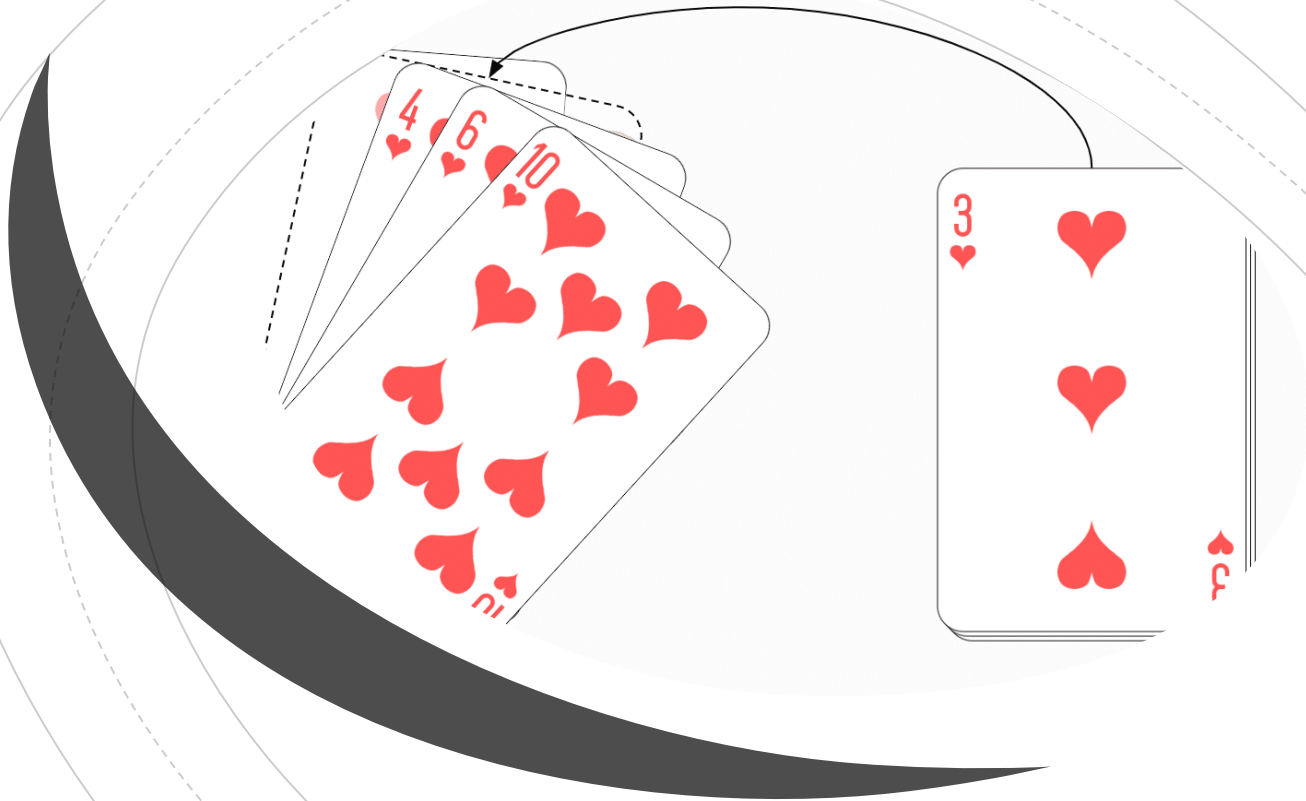
- **Exemple :**
 - On poursuit jusqu'à avoir parcouru N-1 éléments



TRI TERMINE



Tri par insertion





TRI PAR INSERTION

- On assimile souvent ce tri au tri que l'on réalise avec des cartes.
- L'idée ici de réaliser une série de permutations de la droite vers la gauche.
- On continue de permuter tant que la valeur de droite est strictement inférieure à la valeur de gauche.

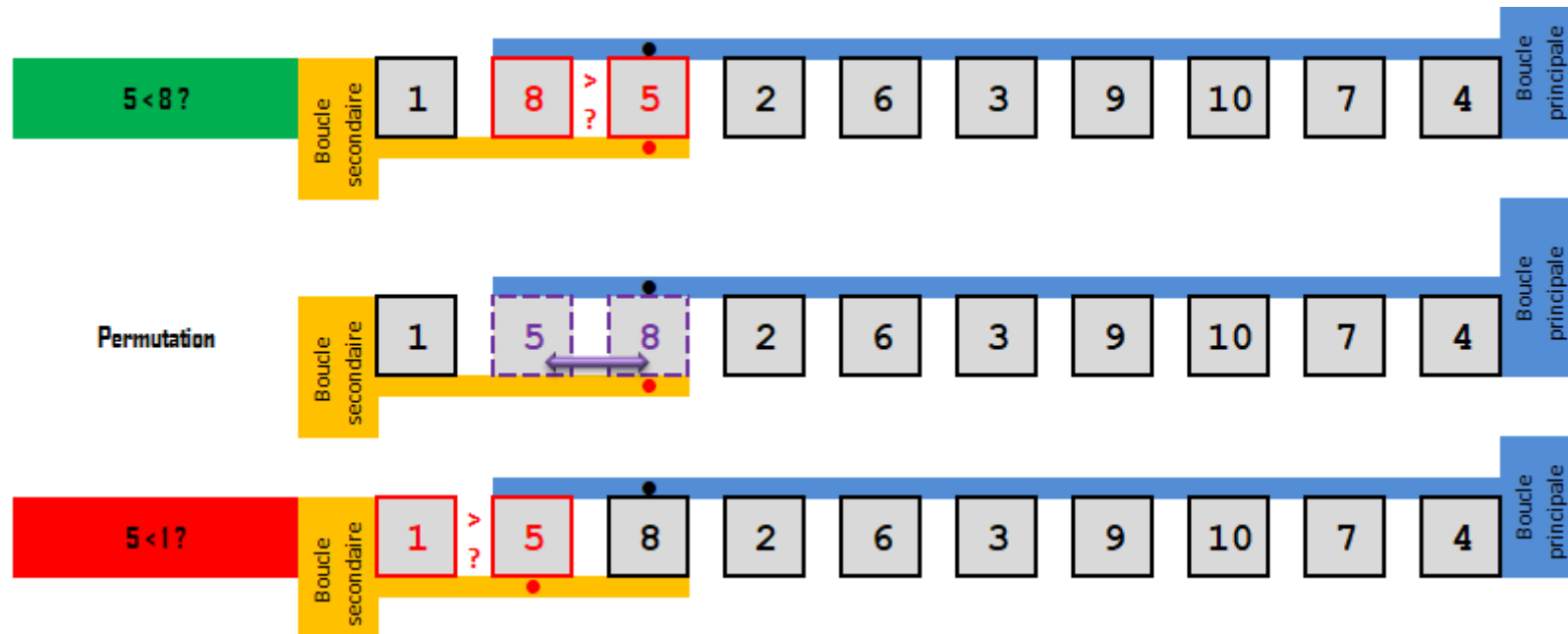
TRI PAR INSERTION

- Exemple :



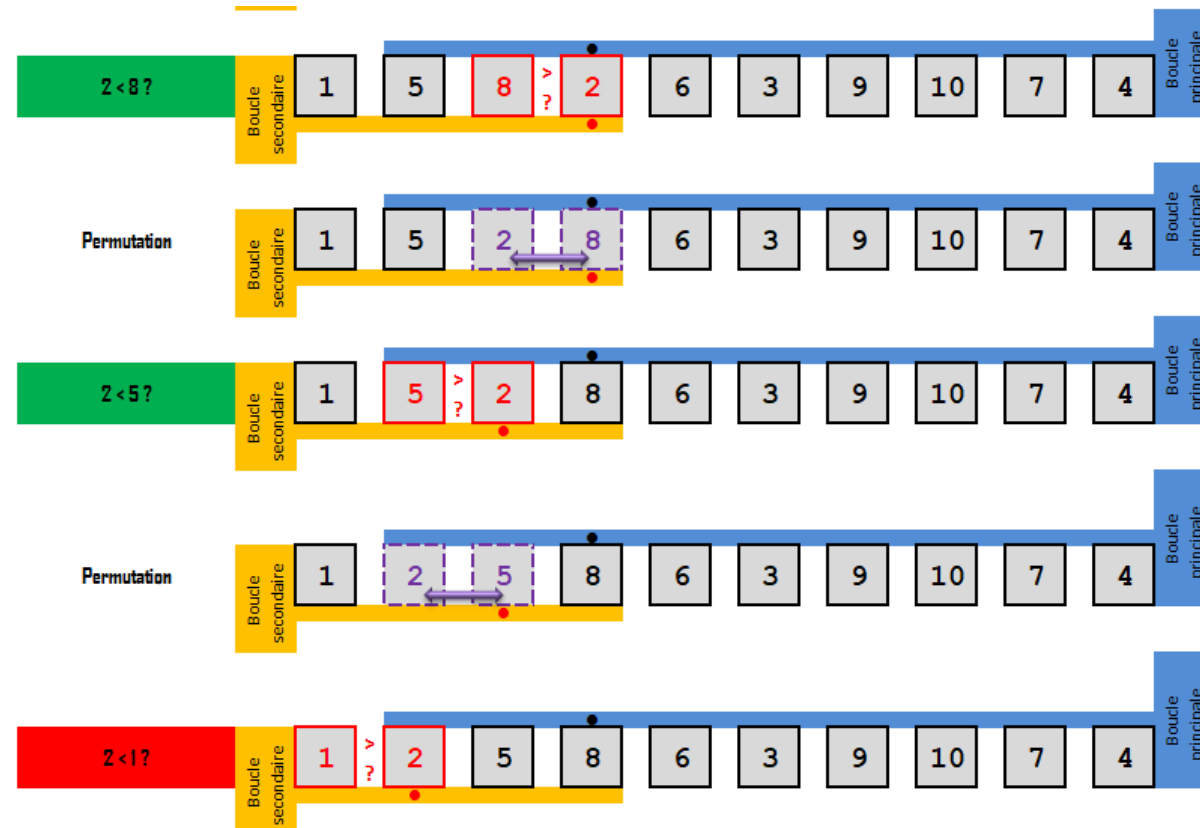
TRI PAR INSERTION

- Exemple :



TRI PAR INSERTION

- Exemple :



TRI PAR INSERTION

- **Exemple :**
- On poursuit ainsi jusqu'à que la boucle principale aie parcouru les éléments de 1 à N.



Tri par à bulles

BONUS CADOR



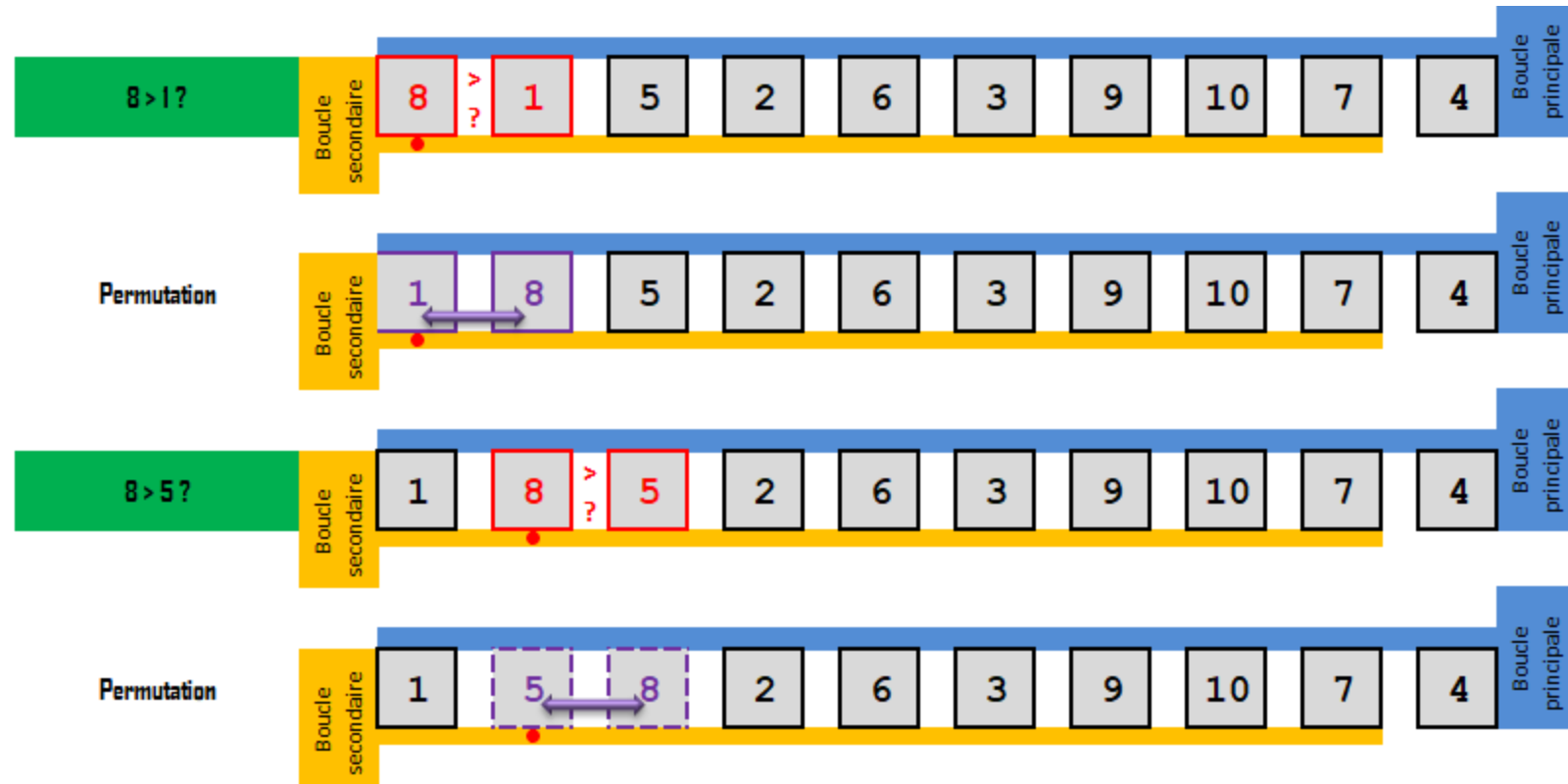
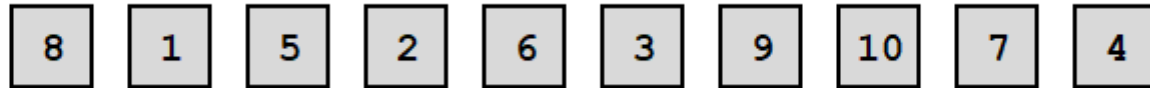
TRI À BULLES.

- Le tri à bulles ressemble au tri à insertion, sauf que l'on fait permuter les valeurs de la gauche vers la droite.
- La valeur montante est appelée bulle.
- Si notre bulle rencontre une plus grosse bulle durant le parcours de la boucle secondaire, alors c'est cette plus grosse bulle qui poursuit le cycle jusqu'à atteindre son rang.
- On s'arrête de trier quand on réalise un parcours sans qu'aucune bulle ne bouge.

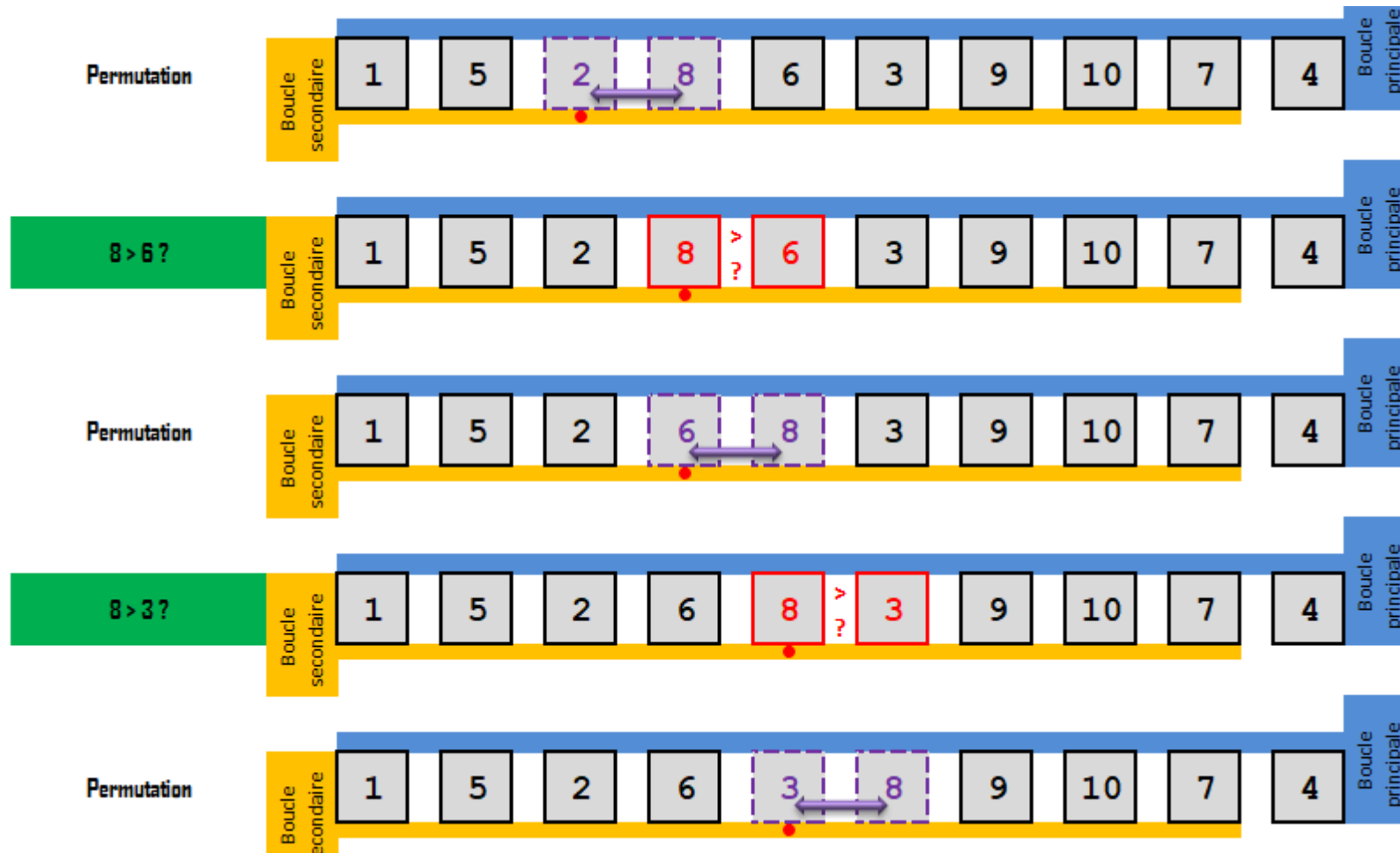
TRI À BULLES.

- Exemple :

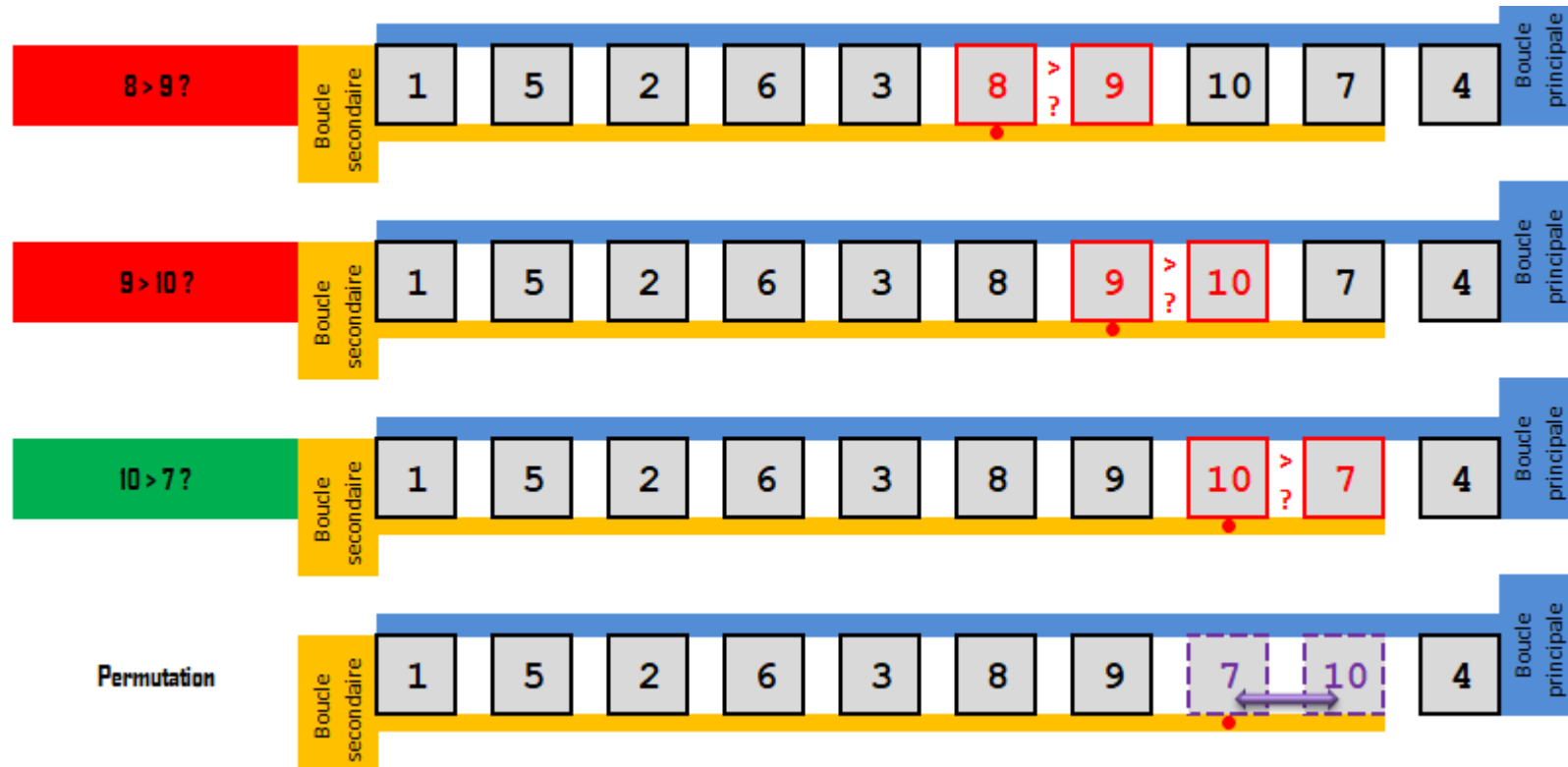
Liste de départ



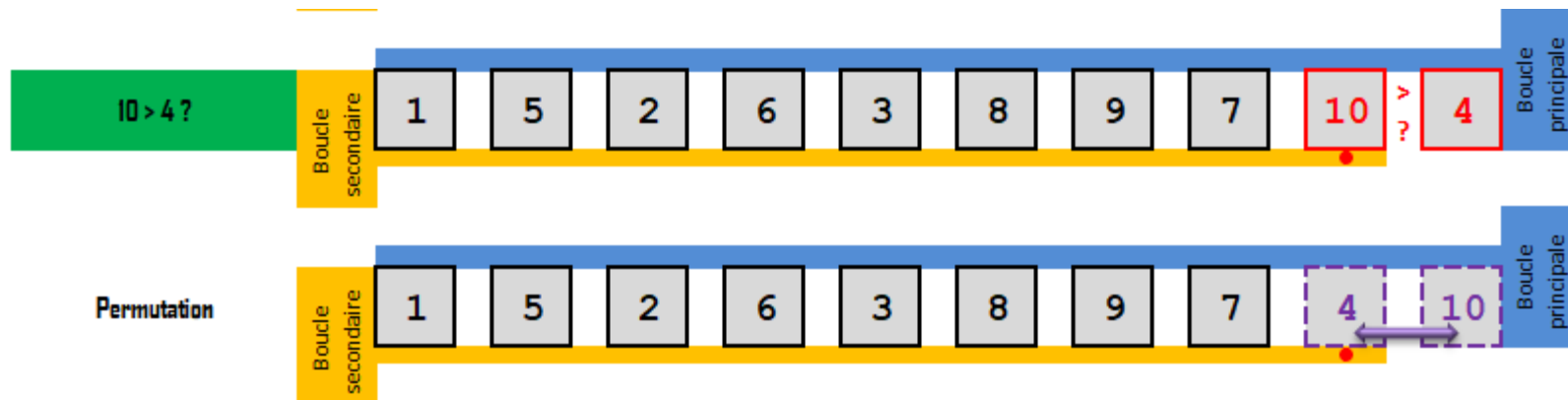
TRI À BULLES.



TRI À BULLES.



TRI À BULLES.



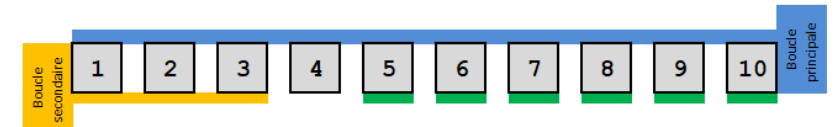
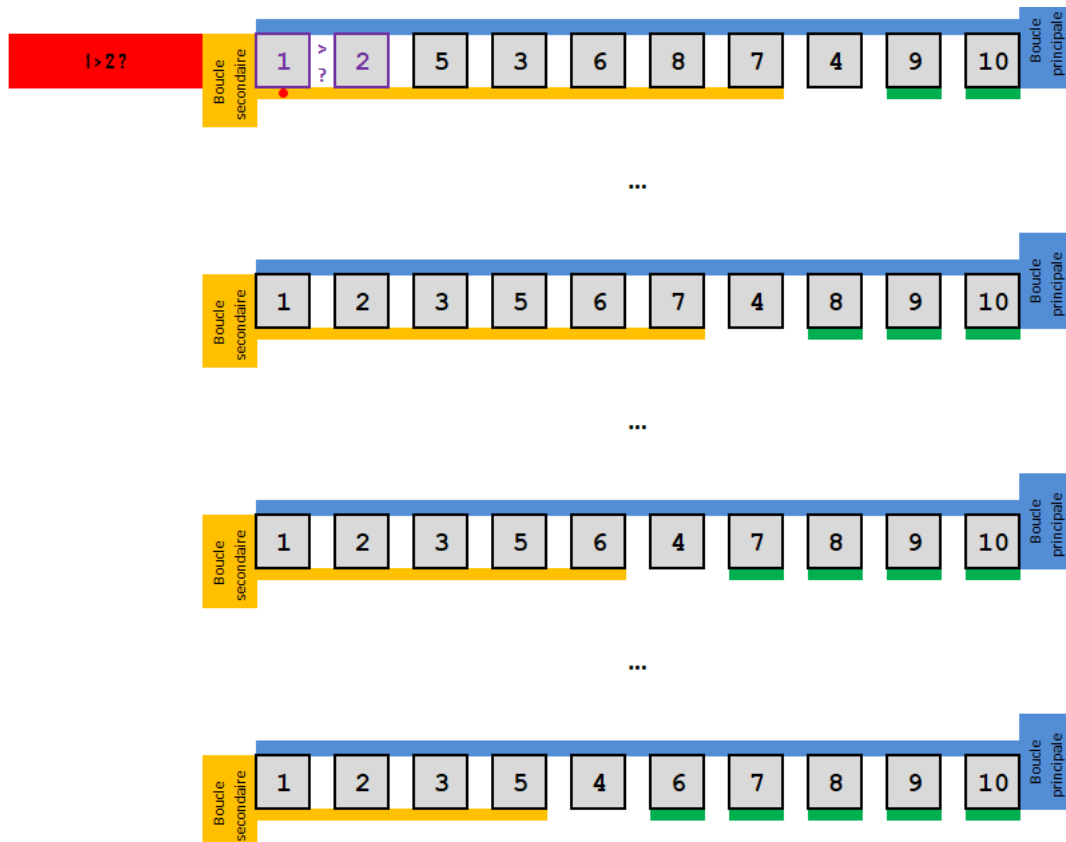
On recommence pour faire remonter la bulle suivante !

TRI À BULLES.



On recommence pour faire remonter la bulle suivante !

TRI À BULLES.



L'algorithme réalise un dernier tour sans faire de permutation, donc le tri est terminé !



FIN